

Stock Market Index Direction Prediction Using Limit Order Book Data

Joseph Raj ¹ Sumanth G.M ² Abhishek Kumar³ Kumara Swamy ⁴ Raghav ⁵ Saagar ⁶

¹Email: aaaaaa@gmail.com *GitHub*: <https://github.com/> *LinkedIn*: <https://www.linkedin.com/>

²Email: aaaaaa@gmail.com *GitHub*: <https://github.com/> *LinkedIn*: <https://www.linkedin.com/>

³Email: aaaaaa@gmail.com *GitHub*: <https://github.com/> *LinkedIn*: <https://www.linkedin.com/>

⁴Email: aaaaaa@gmail.com *GitHub*: <https://github.com/> *LinkedIn*: <https://www.linkedin.com/>

⁵Email: darapaneni@gmail.com *GitHub*: <https://github.com/darapaneni> *LinkedIn*: <https://www.linkedin.com/in/darapaneni/>

⁶Email: aaaaaaaa@gmail.com *GitHub*: <https://github.com/> *LinkedIn*: <https://www.linkedin.com/>

Contents

1	Introduction and Literature Survey	1
1.1	Background and Context	1
1.2	Motivation and Importance of the Study	1
1.3	Problem Definition and Scope	2
1.4	Research Objectives and Questions	2
1.5	Expected Contributions	3
1.6	Review Methodology and Source Selection Criteria	3
1.7	Survey of Existing Approaches	4
1.7.1	Early Solutions and Classical Techniques	4
1.7.2	Modern Approaches and Evolving Trends	4
1.7.3	Emerging Models and Hybrid Frameworks	4
1.8	Comparative Evaluation of Techniques	5
1.9	Dataset and Benchmark Overview	5
1.10	Tools, Frameworks, and Experimental Protocols	5
1.11	Evaluation Metrics Used in Literature	6
1.12	Limitations of Prior Work	6
1.13	Research Gaps and Open Questions	6
1.14	Innovation and Conceptual Synthesis	7
1.15	Visualizations and Knowledge Summaries	7
1.16	Academic Integrity and Citation Style	7
1.17	Summary of the Chapter	7
1.18	Structure of the Report	7
2	Foundations	8
2.1	Introduction to the Foundations Chapter	8
2.2	Theoretical Underpinnings and Conceptual Framework	8
2.2.1	Market Microstructure Theory	8
2.2.2	The Efficient Market Hypothesis and Its Limits	9
2.3	Conceptual Framework	9
2.3.1	Feature Representation	9
2.3.2	Baseline Models	9
2.3.3	Proposed Mamba-LOB Model	10
2.3.4	Training and Evaluation	10
2.3.5	Conclusion	10
2.4	Terminology and Notational Conventions	10
2.4.1	Terminology	11
2.4.2	Notational Conventions	12
2.4.3	Class Labels	12
2.5	Overview of Problem-Solving Paradigms	12

2.5.1	Data-Driven Learning Paradigm	12
2.5.2	Supervised Learning Paradigm	13
2.5.3	Sequential Learning Paradigm	13
2.5.4	Deep Learning Paradigm	13
2.5.5	State Space Modeling Paradigm	14
2.5.6	Comparative Learning Paradigm	14
2.5.7	Optimization Paradigm	14
2.5.8	Experimental and Ablation Paradigm	15
2.6	Algorithmic Foundations	15
2.6.1	Data Preprocessing Algorithms	15
2.6.2	Deep Learning Foundations	16
2.6.3	DeepLOB Architecture	16
2.6.4	Attention Mechanism Foundation	17
2.6.5	Bilinear Normalization Foundation	17
2.7	Classification Algorithm	17
2.7.1	Optimization Algorithms	18
2.7.2	Evaluation Algorithms	18
2.8	Modern Algorithmic Extensions	18
2.9	Model Design Principles	19
2.10	Model Architectures and System Components	19
2.11	Input Processing Layer	20
2.12	Core Processing Unit	20
2.13	Output Generation Layer	20
2.14	Data Preprocessing and Representation	21
2.15	Feature Engineering and Selection Techniques	21
2.16	Experimental Setup and Design Considerations	21
2.17	Evaluation Metrics and Justification	21
2.18	Domain-specific Constraints and Assumptions	22
2.19	Interpretability and Explainability Foundations	22
2.20	Implementation Toolchains (Conceptual View)	22
2.21	Visualization Techniques and Diagrammatic Representation	22
2.22	Ethical, Legal, and Societal Implications (ELSI)	23
2.23	Challenges and Limitations in Theory or Practice	23
2.24	Lessons from Iterative Development and Peer Feedback	23
2.25	Responsible Use of Generative AI in Foundations Work	23
2.26	Summary of the Chapter	24
3	Interim Report	24
3.1	Chapter Overview	24
3.2	Dataset Acquisition and Preprocessing	24
3.2.1	Dataset Description	24
3.2.2	Data Quality Checks	24
3.2.3	Exploratory Data Analysis (EDA)	24

3.2.4	Pre-processing Pipelines	24
3.3	Feature Engineering	24
3.3.1	Feature Identification Techniques	24
3.3.2	Dimensionality Reduction (if applicable)	24
3.3.3	Feature Impact Observations	24
3.4	Technical Implementation	24
3.4.1	Tools and Technology Stack	24
3.4.2	System Setup and Environment Configuration	24
3.4.3	Modular Code-base Description	24
3.5	Baseline Model Development	25
3.5.1	Baseline Algorithm Description	25
3.5.2	Training Setup and Parameters	25
3.5.3	Early Performance Metrics	25
3.5.4	Output Visualization and Interpretation	25
3.6	Preliminary Analysis and Observations	25
3.6.1	Result Analysis and Discussion	25
3.6.2	Error Profiling and Bias Detection	25
3.6.3	Lessons Learned	25
3.7	Originality and Innovation in Approach	25
3.7.1	Creative Framing of the Problem	25
3.7.2	Experimental Innovations	25
3.8	Risk Assessment and Project Management	25
3.8.1	Progress Tracker vs. Original Timeline	25
3.8.2	Bottlenecks and Delays	25
3.8.3	Risk Mitigation Strategies	25
3.9	Experimental Reproducibility	25
3.9.1	Data Splits and Control Measures	25
3.9.2	Random Seed Control	25
3.10	Scalability and Efficiency Considerations	26
3.10.1	Memory, Time, and Resource Profiling	26
3.10.2	Scalability Constraints	26
3.11	Ethical Considerations and Responsible AI Use	26
3.11.1	Generative AI Usage Disclosure	26
3.11.2	Bias and Fairness in Dataset	26
3.12	Feedback Incorporation and Continuous Improvement	26
3.12.1	Peer and Mentor Feedback Summary	26
3.12.2	Reflection and Learning Journey	26
3.13	Chapter Summary and Next Steps	26

1 Introduction and Literature Survey

1.1 Background and Context

Financial markets form the backbone of modern economies, facilitating efficient capital allocation, risk distribution, and continuous price discovery. Stock market prediction has evolved significantly over the past decades. Early forecasting models relied on statistical time-series approaches applied to closing prices.

Over half of global exchanges now operate purely on LOB-based systems, including major exchanges in Paris, Tokyo, Toronto, Sydney, Helsinki, Hong Kong, and Shenzhen, while established exchanges such as the NYSE, NASDAQ, and London Stock Exchange operate hybrid LOB systems. The expansion of High-Frequency Trading (HFT) has amplified both the volume and velocity of data generated within LOBs events now occur at microseconds granularity, producing millions of structured data points per trading session per symbol.

For market indices such as NIFTY 50 India's premier equity index managed by the National Stock Exchange (NSE) the LOB of index futures contracts encapsulates the collective sentiment of a broad market basket of 50 large-cap stocks.

1.2 Motivation and Importance of the Study

Limited research exists on applying LOB based deep architectures to emerging markets such as Indian Markets, here the trading and liquidity are very dynamic and differ significantly from foreign markets. Accurate prediction of index direction is still a complex problem to market due to:

- * Non-Stationary
- * Stochastic Volatility (Randomness)
- * Non-Linear Dependencies
- * Rapidly Evolving Supply Demand mechanisms

Traditionally statistical models (ex: - ARIMA) rely primarily on historical price series and fail to capture granular order book structure. Currently, recent deep learning approaches applied to LOB data predominantly focus on developed markets and rely on discretion time windows, which may ignore event-driven microstructure characteristics.

The study specifically targets Indian Stock Market Indices, addressing a significant geographical research gap in LOB depth interactions. Financial market operates in event-driven time, not uniform clock time. Limit Order Book (LOB) updates occur whenever:

- * A new Limit Order is Placed
- * A Market Order is executed
- * An Order is canceled
- * A price level is modified

Most existing deep learning models convert this irregular stream into fixed time intervals (e.g., 1 second snapshots). These approaches are computationally convenient, this discretionary:

- * Loses intr-interval dynamics
- * Blurs microstructure reactions
- * Introduces sampling bias

* Ignores burst liquidity events.

To address this, the proposed framework introduces event-sensitive feature engineering, where features are constructed based on LOB state transitions rather than fixed timestamps. Unlike existing CNN LSTM hybrid models, the proposed framework emphasizes attention-driven sequence modeling to better capture cross-level dependencies within the order book. All depth levels and all past events can dynamically influence the current prediction.

1.3 Problem Definition and Scope

Most existing deep learning models convert this irregular stream into fixed time intervals (e.g., 1 second snapshots). These approaches are computationally convenient, this discretionary:

- * Loses intr-interval dynamics
- * Blurs microstructure reactions
- * Introduces sampling bias
- * Ignores burst liquidity events.

To address this, the proposed framework introduces event-sensitive feature engineering, where features are constructed based on LOB state transitions rather than fixed timestamps. Unlike existing CNN LSTM hybrid models, the proposed framework emphasizes attention-driven sequence modelling to better capture cross-level dependencies within the order book. All depth levels and all past events can dynamically influence the current prediction. The scope includes:

1. High frequency LOB data consists of:
 - * Top-k bid prices and volumes
 - * Top-k ask prices and volumes
 - * Event timestamps
 - * Event types (limit order, market order, cancellation)
2. Microstructure feature engineering (spread, imbalance, order flow)
3. Development of hybrid CNN Transformer models
4. Benchmarking against ARIMA, HMM, and CNN LSTM
5. Multi-horizon and regime-based evaluation

1.4 Research Objectives and Questions

Objectives

- Construct event-aligned LOB datasets similar to the approach in Deep Hybrid Models for Forecasting Stock Mid prices from the High-Frequency Limit Order Book.
- Engineer imbalance-aware features inspired by Continuous-time Modeling of Financial Returns Based on Limit Order Book Data, particularly Base Imbalance.
- Develop a Transformer-enhanced architecture to overcome limitations of CNN LSTM hybrids

- **Benchmark Construction:** Construct and reprocess a high-frequency LOB dataset for NIFTY 50 index futures from NSE, enabling reproducible research on Indian market microstructure.

Research Questions:

1. Does attention-based modeling outperform CNN LSTM hybrids?
2. Can Limit Order Book data from NIFTY 50 index futures reliably predict the short-term directional movement of the index mid-price at horizons $H = 1, 5$, and 10 events?
3. What are the most informative features within the LOB for index direction prediction: raw price-level features, volume imbalance indicators, or derived time-sensitive features?

1.5 Expected Contributions

Contributions :

- **Benchmark Dataset:** Creation of a structured, preprocessed NIFTY 50 index futures LOB dataset from NSE the first of its kind for the Indian market. Fills the geographic and market-type gap identified across all reviewed literature.
- **Extend LOB-based modeling** to an emerging market context
- **Provide comprehensive multi-regime benchmarking** absent in existing works
- **Comparative Empirical Study:** Rigorous comparison of 8+ model classes on the NIFTY LOB dataset under standardized experimental protocols, providing a reusable and citable benchmark for future Indian market ML research.

1.6 Review Methodology and Source Selection Criteria

The literature survey includes:

- Classical statistical approaches (ARIMA)
- Deep learning LOB models (CNN, LSTM, CNN LSTM hybrids)
- Continuous-time microstructure models

Key selection criteria:

- Use of high-frequency financial data
- Inclusion of LOB structure
- Empirical validation
- Hybrid modelling innovation

Primary references include:

- (a) Deep Hybrid Models for Forecasting Stock Mid prices from the High-Frequency Limit Order Book

- (b) Continuous-time Modeling of Financial Returns Based on Limit Order Book Data
- (c) Analysis of Enhanced Hidden Markov Models for Improved Stock Market Price Forecasting and Prediction
- (d) Stock Market Price Analysis and Prediction of Financial Industry Based on the Time Series Method
- (e) Rui Liu (2024), "Mid-Price Forecasting in Limit Order Books with Gaussian Process Models," Tampere University MSc Thesis - the direct methodological predecessor synthesizing 50 key references.

1.7 Survey of Existing Approaches

1.7.1 Early Solutions and Classical Techniques

The earliest quantitative approaches to financial prediction used parametric statistical models that assumed specific data generating processes. The study **Stock Market Price Analysis and Prediction of Financial Industry Based on the Time Series Method** compares **AR, MA, ARMA, and ARIMA**, concluding that **ARIMA performs better than basic linear models but fails under nonlinear dynamics**.

Similarly, Analysis of **Enhanced Hidden Markov Models** for Improved Stock Market Price Forecasting and Prediction introduces **improved HMM structures and hybrid HMM ARIMA combinations**. While achieving strong validation performance, these methods:

- Depend on historical price series
- Do not use LOB depth data
- Are sensitive to regime misclassifications

1.7.2 Modern Approaches and Evolving Trends

Deep Hybrid Models for Forecasting Stock Mid prices from the High-Frequency Limit Order Book introduces CNN, LSTM, and hybrid CNN-LSTM architectures applied to NASDAQ LOB data. The study demonstrates:

- Spatial extraction via CNN
- Temporal modeling via LSTM
- Improved mid-price movement prediction

However, it relies on time-aggregated snapshots and does not incorporate attention mechanisms.

1.7.3 Emerging Models and Hybrid Frameworks

Continuous-time Modeling of Financial Returns Based on Limit Order Book Data proposes event-driven modeling and introduces Base Imbalance as a superior microstructure regressor. This motivates integrating:

- Event-sensitive feature engineering
- Imbalance-aware signals
- Advanced sequence modeling (e.g., Transformer architectures)

The present research is based on these foundations.

1.8 Comparative Evaluation of Techniques

Table 1. Comparison of LOB Modeling Approaches

Approach	Strength	Limitation	Example Paper
ARIMA	Linear forecasting	Cannot model non-linear LOB	Time Series Method Paper
HMM	Regime modeling	No microstructure data	Enhanced HMM Paper
CNN	Depth-level learning	Local interaction only	Deep Hybrid Models Paper
CNN LSTM	Spatial + temporal	Sequential bottleneck	Deep Hybrid Models Paper
Continuous-Time Model	Event-driven	Linear assumption	Continuous-Time LOB Paper

1.9 Dataset and Benchmark Overview

Proposed NIFTY 50 Index Futures LOB Dataset:

Source: NSE (National Stock Exchange of India) Level-2 NIFTY 50 index futures market data via authorized data vendor

Target Period: Multiple months of trading data capturing different market regimes

Features: 40 base features (10-level bid/ask prices and volumes) + 14 derived features (spread, imbalance ratios, order flow indicators) = up to 54 features

Trading Hours Filter: 9:15 AM to 3:30 PM IST, with opening (9:15 to 9:30) and closing (3:20 to 3:30) auction periods filtered to isolate continuous trading

Contribution: First publicly shared Indian equity index futures LOB ML benchmark dataset

1.10 Tools, Frameworks, and Experimental Protocols

Software Tools and Libraries

- Python 3.10+: Primary development language - full access to scientific ML ecosystem.
- Scikit-learn: GP Regression and Classification (GPR/GPC) using Laplace approximation for multi-class GPC inference.
- TensorFlow / PyTorch: Implementation of CNN, LSTM, and TABL architectures. PyTorch preferred for research flexibility and dynamic graph building
- NumPy / Pandas: Data manipulation, rolling window feature construction, LOB reprocessing pipelines, and statistical analysis.
- Google Colab / TPU: Large-scale parallel GP training. This project will use equivalent GPU/TPU resources.
- SciPy: Statistical significance testing - Kruskal-Wallis ANOVA.
- Matplotlib / Seaborn: Visualization - confusion matrices, entropy histograms, confidence interval plots, performance comparison charts.

Experimental Protocol:

- 1. **Data Split:** Chronological 70/30 first 7 trading days for training, last 3 for testing. Last 10 percent of training used as validation.

- **2. Normalization:** Z-score per feature column, computed from training statistics only and applied to test data
- **3. Input Construction:** Rolling window of $W = 10$ consecutive events, yielding input shape $[10 \times 40]$ per sample.
- **4. Class Labeling:** Percentage mid-price change formula with threshold $\hat{\tau}$ to define Up/Down/Stationary boundaries.
- **5. Ensemble Strategy:** Training data split into chunks of 8,000 samples for parallel GP training.
- **6. Evaluation Horizons:** $H = 1, 5$, and 10 future events for classification; $H = 10,000, 13,000, 15,000$ events for regression.

1.11 Evaluation Metrics Used in Literature

Table 2. Evaluation Metrics for LOB Modeling

Metric	Formula / Definition	Why It Matters
F1 Score	$2 \times \frac{P \times R}{P + R}$	Robust to class imbalance - stationary class dominates 70-74% of samples.
Accuracy (%)	Correct / Total	Overall metric but misleading under class imbalance; used as a supplementary metric.
Precision (%)	$TP / (TP + FP)$	Measures prediction reliability for each direction class.
Recall (%)	$TP / (TP + FN)$	Measures sensitivity - important for detecting rare directional changes.
Win-Rate	Accuracy excluding stationary predictions	Most actionable trading metric - measures Up vs Down discrimination ability.
Entropy $H(p)$	$-\sum p(y x) \log p(y x)$	Measures model confidence calibration. Lower values indicate more confident predictions.
R^2 / MSE	Regression goodness-of-fit	Used for mid-price value prediction tasks to measure error and fit.

1.12 Limitations of Prior Work

Across the reviewed papers:

- No emerging market application.
- Limited event-driven deep architectures.
- No Transformer-based modeling.
- Limited volatility-regime robustness evaluation.

1.13 Research Gaps and Open Questions

- Can attention-based models outperform CNN LSTM in LOB prediction?
- Does Base Imbalance improve deep learning stability?
- How do LOB-based models generalize in emerging markets?
- Can event-sensitive modeling reduce overfitting?

1.14 Innovation and Conceptual Synthesis

The present work synthesis's:

- Classical benchmarking (ARIMA, HMM)
- CNN spatial extraction (Deep Hybrid Models)
- Event-sensitive imbalance modeling (Continuous-time LOB study)
- Transformer-based temporal attention to construct a hybrid imbalance-aware deep learning framework.

1.15 Visualizations and Knowledge Summaries

This includes:

- Comparative methodology tables
- Hybrid architecture diagrams
- Data pipeline schematics
- Regime evaluation flowcharts

These summaries evolution from statistical to deep hybrid models.

1.16 Academic Integrity and Citation Style

All referenced studies are properly d using IEEE/APA format in the final bibliography. Conceptual explanations are original synthesis while empirical findings are attributed to the respective works.

1.17 Summary of the Chapter

This literature survey reviewed classical ARIMA and HMM models, deep hybrid CNN LSTM architectures, and continuous-time LOB modeling approaches.

While deep models demonstrate improved performance, gaps remain in emerging market application, attention-based modeling, and regime robustness. These motivate the proposed research.

1.18 Structure of the Report

- Chapter 1: Introduction
- Chapter 2: Literature Review
- Chapter 3: Proposed Methodology
- Chapter 4: Experimental Evaluation
- Chapter 5: Results and Discussion
- Chapter 6: Conclusion and Future Work

2 Foundations

2.1 Introduction to the Foundations Chapter

This chapter provides a high-level overview and establishes the theoretical and conceptual foundation of the capstone project, whose central objective is the prediction of intraday price movements of the **Nifty 50** index using high-frequency **Limit Order Book (LOB)** data sourced from the National Stock Exchange of India (NSE).

The Nifty 50 is India's leading large-cap equity index, comprising fifty stocks that collectively represent approximately 65% of the free-float market capitalization of all equities listed on the NSE. Its LOB encapsulates real-time tick-by-tick supply and demand dynamics at multiple price levels, offering a rich micro structural signal that is demonstrably predictive over short horizons .

Previous work has established that deep learning architectures –particularly those combining convolution and recurrent layers – significantly outperform classical statistical models on LOB-based directional forecasting tasks. This project extends those findings to the Indian equity market, targeting the Nifty 50 mid-price movement direction over multiple prediction horizons.

The remainder of this chapter is organized as follows. presents the theoretical underpinnings,survey algorithmic approaches from rule-based methods to modern Transformer architectures,describe model design principles and system components, detail data handling, specify the experimental framework, address domain constraints, interpret ability, tool chains, visualization, and ethics, acknowledges limitations, and synthesis's the chapter.

2.2 Theoretical Underpinnings and Conceptual Framework

This section presents the fundamental theories, principles, and scientific laws that support the proposed system and methodology.

2.2.1 Market Microstructure Theory

Market microstructure theory examines the mechanisms by which latent investor trading intentions are translated into observable prices and quantities through explicit trading rules and institutions

The continuous double auction, the dominant mechanism on modern electronic exchanges including the NSE, maintains a LOB that records all outstanding limit orders at each discrete price level. Two sides of the book exist at all times:

- **Bid side:** buy limit orders, sorted in descending price order; the highest bid is the *best bid* P_1^b .
- **Ask side:** sell limit orders, sorted in ascending price order; the lowest ask is the *best ask* P_1^a .

The *mid-price* p_{mid} serves as the primary forecasting target because it is less susceptible to microstructural noise than the last transaction price :

$$p_{\text{mid}}(t) = \frac{P_1^a(t) + P_1^b(t)}{2}. \quad (2.1)$$

The *bid-ask spread* $S(t) = P_1^a(t) - P_1^b(t)$ is a direct measure of market liquidity and has been exploited for pairs-trading strategies using high-frequency LOB data. The *depth* at each level—the volume available—encodes short-term price pressure.

2.2.2 The Efficient Market Hypothesis and Its Limits

The semi-strong form of the Efficient Market Hypothesis (EMH) asserts that all publicly available information is immediately reflected in asset prices, implying no risk-adjusted excess returns from historical price data. However, empirical high-frequency studies consistently document short-lived predictability at sub-second to minute horizons, attributable to order-flow imbalances, inventory management by market makers, and latency-driven arbitrage. Deep learning models trained on LOB snapshots exploit precisely these transient inefficiencies, and their documented predictive power motivates this project.

2.3 Conceptual Framework

The conceptual framework of the proposed project explains the overall process used for stock price movement prediction using Limit Order Book (LOB) data. The framework combines financial market data processing with deep learning models to improve prediction accuracy and efficiency.

The framework begins with collecting data from the FI-2010 benchmark dataset and optional NSE live market data. The collected data contains bid prices, ask prices, bid volumes, and ask volumes from multiple levels of the order book.

Data Preprocessing

The raw data is processed before training the models. The preprocessing steps include:

- Data cleaning
- Missing value handling
- Z-score normalization
- Sliding window sequence generation
- Label verification

The processed data is converted into sequential input format for deep learning models.

2.3.1 Feature Representation

The framework uses multiple order book features such as:

- Bid prices
- Ask prices
- Bid volumes
- Ask volumes

These features help the model understand market behavior and price movement patterns.

2.3.2 Baseline Models

The project includes baseline models for comparison:

- DeepLOB
- MLPLOB

These models are used to compare the performance of the proposed Mamba-LOB model.

2.3.3 Proposed Mamba-LOB Model

The proposed Mamba-LOB architecture is based on the Mamba State Space Model (SSM). The model processes sequential LOB data efficiently and captures long-term temporal dependencies.

The architecture includes:

- Input Layer
- Linear Projection Layer
- Mamba Blocks
- Attention Layer (optional)
- Fully Connected Layer
- Softmax Output Layer

The model predicts three market movement classes:

- Upward Movement
- Downward Movement
- Stationary Movement

2.3.4 Training and Evaluation

The framework uses supervised learning for model training. The training process uses:

- CrossEntropy Loss
- AdamW Optimizer
- Early Stopping

The model performance is evaluated using:

- Accuracy
- Weighted F1-Score
- Validation Loss

2.3.5 Conclusion

The conceptual framework provides a complete pipeline for financial market prediction using deep learning techniques. The proposed Mamba-LOB model aims to improve prediction performance, computational efficiency, and sequential learning capability for high-frequency trading data.

2.4 Terminology and Notational Conventions

This section describes the important terms, symbols, and notations used in the proposed Mamba-LOB implementation framework. These conventions help maintain consistency and improve understanding throughout the project.

2.4.1 Terminology

- **Limit Order Book (LOB):** A digital record containing buy and sell orders for a financial asset at different price levels.
- **High-Frequency Trading (HFT):** A trading method that processes large numbers of market transactions within very short time intervals.
- **FI-2010 Dataset:** A benchmark financial dataset containing 10-level order book data used for stock price prediction research.
- **Order Book Depth:** The number of bid and ask price levels available in the market order book.
- **Bid Price:** The maximum price buyers are willing to pay for a stock.
- **Ask Price:** The minimum price sellers are willing to accept for a stock.
- **Sequence Length:** The number of continuous order book events used as input for the model.
- **Sliding Window:** A preprocessing method used to generate sequential input samples from time-series data.
- **Normalization:** A preprocessing technique used to scale input features for stable model training.
- **Z-score Normalization:** A statistical normalization technique that standardizes data using mean and standard deviation.
- **DeepLOB:** A deep learning baseline model combining CNN and LSTM layers for LOB prediction.
- **MLPLOB:** A multilayer perceptron-based baseline model used for comparison.
- **Mamba-LOB:** The proposed prediction model based on the Mamba State Space Model architecture.
- **State Space Model (SSM):** A sequential modeling approach used for capturing long-term temporal dependencies efficiently.
- **Attention Mechanism:** A deep learning component that helps the model focus on important features in the input sequence.
- **Softmax Function:** An activation function used for multi-class classification probability prediction.
- **Epoch:** One complete iteration of training over the entire dataset.
- **Batch Size:** The number of training samples processed in one iteration.
- **CrossEntropy Loss:** A loss function used for classification tasks.
- **AdamW Optimizer:** An optimization algorithm used for updating neural network weights efficiently.
- **Weighted F1-Score:** A performance evaluation metric used for imbalanced classification problems.

2.4.2 Notational Conventions

The mathematical symbols and notations used in this project are listed below.

Notation	Description
X	Input sequence data
Y	Output feature representation
L	Sequence length of the input order book data
F	Number of input features
B	Batch size used during training
d_{model}	Feature projection dimension in Mamba architecture
d_{state}	State size used in the Mamba State Space Model
d_{conv}	Convolution width parameter in Mamba blocks
H	Prediction horizon
$\hat{y}$	Predicted output class
W	Weight matrix of the neural network
b	Bias parameter
lr	Learning rate used for optimization
$\mathbb{R}$	Set of real numbers
$X \in \mathbb{R}^{L \times F}$	Input feature matrix with sequence length and feature dimensions

Table 3. Terminology and Notational Conventions

2.4.3 Class Labels

The proposed framework uses a three-class classification approach for stock movement prediction.

Class Label	Meaning
1	Upward Price Movement
2	Stationary Market Condition
3	Downward Price Movement

Table 4. Prediction Class Labels

2.5 Overview of Problem-Solving Paradigms

The proposed Mamba-LOB framework uses multiple problem-solving paradigms from machine learning, deep learning, and financial data analysis to predict short-term stock price movements using Limit Order Book (LOB) data. The framework combines data-driven learning, sequential modeling, and comparative experimentation to improve prediction performance and computational efficiency.

2.5.1 Data-Driven Learning Paradigm

The proposed system follows a data-driven learning approach where the model learns patterns directly from historical market data instead of using manually designed trading rules.

The framework uses:

- FI-2010 benchmark dataset
- Real-time NSE order book data (optional)
- Historical bid and ask information
- Sequential order book events

The collected data is used to train deep learning models for automatic feature learning and market trend prediction.

2.5.2 Supervised Learning Paradigm

The framework uses supervised learning for stock movement classification. In supervised learning, the model learns the relationship between input order book sequences and labeled market movement classes.

The classification categories are:

- Upward Movement
- Downward Movement
- Stationary Movement

The labeled data helps the model learn market behavior and predict future price movements accurately.

2.5.3 Sequential Learning Paradigm

Financial market data is sequential in nature because order book events occur continuously over time. Therefore, the framework uses sequential learning techniques to capture temporal dependencies.

The proposed Mamba-LOB model processes sequential input windows to learn:

- Market trends
- Temporal price changes
- Order flow patterns
- Long-range dependencies

The sequence input is represented as:

$$X \in \mathbb{R}^{L \times F}$$

where:

- L represents sequence length
- F represents feature dimensions

2.5.4 Deep Learning Paradigm

The framework uses deep learning models for automatic feature extraction and high-dimensional data processing.

The deep learning architectures used include:

- DeepLOB
- MLPLOB
- Proposed Mamba-LOB

These models help identify complex nonlinear relationships in financial market data.

2.5.5 State Space Modeling Paradigm

The proposed Mamba-LOB model is based on the State Space Model (SSM) paradigm. State space models are designed for efficient sequence modeling and long-term dependency learning.

The Mamba architecture provides:

- Efficient sequence processing
- Reduced computational complexity
- Better scalability for long sequences
- Improved temporal feature learning

The sequence transformation is represented as:

$$Y = \text{Mamba}(X)$$

where:

- X is the input sequence
- Y is the transformed feature representation

2.5.6 Comparative Learning Paradigm

The framework follows a comparative learning approach by evaluating the proposed Mamba-LOB model against existing baseline models.

The comparison models include:

- DeepLOB
- MLPLOB
- TLOB (optional)

This paradigm helps measure the effectiveness of the proposed architecture using standard evaluation metrics.

2.5.7 Optimization Paradigm

The training framework uses optimization techniques to improve model learning and reduce prediction error.

The optimization components include:

- CrossEntropy Loss Function
- AdamW Optimizer
- Learning Rate Scheduling
- Early Stopping

These techniques improve training stability and model convergence.

2.5.8 Experimental and Ablation Paradigm

The framework also follows an experimental research paradigm using ablation studies.

The ablation experiments analyze:

- Attention mechanism effectiveness
- Sequence length variations
- Normalization methods
- Model architecture performance

These experiments help identify the contribution of individual components in the proposed framework.

2.6 Algorithmic Foundations

The proposed Mamba-LOB framework is built on multiple algorithmic foundations from machine learning, deep learning, sequential modeling, and financial data analysis. These algorithms help process high-frequency Limit Order Book (LOB) data and predict short-term stock price movements efficiently.

The framework combines preprocessing algorithms, deep learning architectures, optimization methods, and sequence modeling techniques to improve prediction performance and computational efficiency.

2.6.1 Data Preprocessing Algorithms

The first stage of the framework uses preprocessing algorithms to prepare raw financial data for model training.

Data Cleaning

Data cleaning algorithms remove incomplete and invalid records from the dataset to improve data quality and model reliability.

Sliding Window Algorithm

The sliding window algorithm converts continuous market events into sequential input samples.

The sequence input is represented as:

$$X \in \mathbb{R}^{L \times F}$$

where:

- L represents sequence length
- F represents the number of features

The framework primarily uses a sequence length of $L = 100$.

Z-score Normalization

Z-score normalization standardizes the feature values using mean and standard deviation.

The normalization formula is:

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X represents the original feature value
- μ represents the mean
- σ represents the standard deviation

This normalization improves training stability and reduces feature scale variation.

2.6.2 Deep Learning Foundations

The framework uses deep learning algorithms for automatic feature extraction and sequential market learning.

2.6.3 DeepLOB Architecture

DeepLOB combines Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks.

- CNN layers extract spatial market features
- LSTM layers capture temporal dependencies

This model acts as a baseline architecture for performance comparison.

MLPLOB Architecture

MLPLOB uses a simple multilayer perceptron structure with fully connected neural layers.

The model includes:

- Flatten Layer
- Dense Neural Layers
- Bilinear Normalization (BiN)
- Softmax Classification Layer

MLPLOB provides a lightweight baseline for evaluating the proposed architecture.

Mamba State Space Model Foundation

The core algorithmic foundation of the project is the Mamba State Space Model (SSM).

The Mamba architecture processes long sequential inputs efficiently while maintaining lower computational complexity compared to Transformer models.

The sequence transformation is represented as:

$$Y = \text{Mamba}(X)$$

where:

- X is the input sequence
- Y is the transformed output representation

The Mamba architecture consists of:

- Linear Projection Layer
- Mamba Sequential Blocks
- State Space Modeling Components

- Convolution Operations

The major advantages of Mamba include:

- Efficient long-sequence processing
- Reduced memory usage
- Faster computation
- Improved temporal dependency learning

2.6.4 Attention Mechanism Foundation

The framework optionally integrates Spatial Multi-Head Attention (MHA).

The attention mechanism helps the model focus on important feature relationships within the order book data.

The attention mechanism improves:

- Feature interaction learning
- Spatial dependency analysis
- Important pattern identification

2.6.5 Bilinear Normalization Foundation

The framework uses Bilinear Normalization (BiN) to stabilize feature distributions and improve model convergence.

BiN helps:

- Reduce feature imbalance
- Improve training stability
- Handle market volatility variations

2.7 Classification Algorithm

The final prediction layer uses the Softmax classification algorithm to predict market movement classes.

The prediction function is:

$$\hat{y} = \text{Softmax}(Wx + b)$$

where:

- W represents learned weights
- x represents feature vectors
- b represents bias values

The framework predicts three classes:

- Upward Movement
- Downward Movement
- Stationary Movement

2.7.1 Optimization Algorithms

The training process uses optimization algorithms to minimize prediction error and improve learning performance.

CrossEntropy Loss

CrossEntropy Loss is used for multi-class classification tasks.

The loss function measures the difference between predicted and actual class labels.

AdamW Optimizer

The framework uses the AdamW optimization algorithm for efficient neural network weight updates.

AdamW provides:

- Faster convergence
- Adaptive learning rates
- Improved regularization

Early Stopping

Early stopping prevents overfitting by stopping training when validation performance no longer improves.

2.7.2 Evaluation Algorithms

The framework uses multiple evaluation metrics to measure prediction performance.

The evaluation metrics include:

- Accuracy
- Weighted F1-Score
- Validation Loss

These metrics help compare the proposed Mamba-LOB model with baseline architectures.

2.8 Modern Algorithmic Extensions

Modern algorithmic extensions improve the efficiency, scalability, and prediction capability of traditional financial forecasting models. The proposed framework extends classical deep learning methods using the Mamba State Space Model (SSM), attention mechanisms, and normalization techniques.

The modern extensions used in this project include:

- Mamba State Space sequential modeling
- Spatial Multi-Head Attention
- Bilinear Normalization (BiN)
- Adaptive optimization techniques

These extensions improve long-sequence learning, reduce computational complexity, and enhance prediction accuracy for high-frequency trading data.

2.9 Model Design Principles

The proposed framework follows important software and model design principles for scalable implementation.

- **Modularity:** Separate modules are used for preprocessing, training, evaluation, and visualization.
- **Reusability:** Components such as normalization layers and preprocessing pipelines are reused across multiple models.
- **Adaptability:** The framework supports FI-2010 data and optional NSE live data integration.
- **Scalability:** The architecture supports larger sequence lengths and multiple stock datasets.

2.10 Model Architectures and System Components

The framework consists of the following major components:

1. Data Acquisition Layer
2. Data Preprocessing Layer
3. Feature Representation Layer
4. Baseline Models
5. Proposed Mamba-LOB Model
6. Training and Optimization Layer
7. Evaluation Layer

The proposed Mamba-LOB architecture includes:

- Input Layer
- Linear Projection Layer
- Mamba Sequential Blocks
- Attention Layer (optional)
- Bilinear Normalization Layer
- Fully Connected Layer
- Softmax Output Layer

2.11 Input Processing Layer

The input processing layer handles raw Limit Order Book (LOB) data before model training.

The major operations include:

- Data cleaning
- Missing value handling
- Label verification
- Z-score normalization
- Sliding window generation

The processed input sequence is represented as:

$$X \in \mathbb{R}^{L \times F}$$

where L represents sequence length and F represents feature dimensions.

2.12 Core Processing Unit

The core processing unit of the framework is the Mamba-LOB architecture.

The model processes sequential order book data using:

- State Space Modeling
- Sequential feature learning
- Temporal dependency extraction
- Attention-based feature interaction

The sequence transformation is represented as:

$$Y = \text{Mamba}(X)$$

The framework also includes DeepLOB and MLPLOB as baseline models.

2.13 Output Generation Layer

The output layer produces stock movement predictions using Softmax classification.

The framework predicts:

- Upward Price Movement
- Downward Price Movement
- Stationary Market Condition

The prediction function is:

$$\hat{y} = \text{Softmax}(Wx + b)$$

The generated outputs are evaluated using classification metrics.

2.14 Data Preprocessing and Representation

Data preprocessing improves data quality and training stability.

The preprocessing steps include:

- Data cleaning
- Normalization
- Feature scaling
- Sequence generation
- Data formatting

Rolling z-score normalization is applied to standardize feature distributions.

2.15 Feature Engineering and Selection Techniques

The framework extracts important market features from Limit Order Book data.

The features include:

- Bid prices
- Ask prices
- Bid volumes
- Ask volumes

The framework uses sequential order book windows instead of manually engineered trading indicators.

2.16 Experimental Setup and Design Considerations

The experiments are designed to evaluate the performance of the proposed Mamba-LOB architecture.

The experimental setup includes:

- FI-2010 benchmark dataset
- Multiple stock experiments
- Different prediction horizons
- Baseline model comparison
- Ablation studies

The experiments are executed using Google Colab GPU environments.

2.17 Evaluation Metrics and Justification

The framework uses multiple evaluation metrics for performance analysis.

- Accuracy
- Weighted F1-Score
- Validation Loss

Weighted F1-score is selected because financial datasets are often imbalanced.

2.18 Domain-specific Constraints and Assumptions

The framework considers several domain-specific constraints.

- Limited availability of historical LOB datasets
- High computational requirements for sequence models
- Market volatility variations
- Real-time processing limitations

The framework assumes that historical order book patterns help predict short-term market movements.

2.19 Interpretability and Explainability Foundations

The framework improves interpretability through:

- Comparative analysis
- Attention visualization
- Performance metrics
- Ablation studies

These methods help understand the contribution of different model components.

2.20 Implementation Toolchains (Conceptual View)

The framework uses the following tools and platforms:

- Python
- PyTorch
- Google Colab
- AWS EC2
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

These tools support model training, preprocessing, visualization, and experimentation.

2.21 Visualization Techniques and Diagrammatic Representation

Visualization techniques help analyze model performance and experimental results.

The framework uses:

- Flowcharts
- Model architecture diagrams
- Confusion matrices
- Line charts
- Bar graphs

These visualizations improve result interpretation and presentation clarity.

2.22 Ethical, Legal, and Societal Implications (ELSI)

The framework considers ethical and responsible AI principles.

The considerations include:

- Data privacy
- Fair model evaluation
- Transparent experimentation
- Responsible financial prediction usage

The project uses publicly available benchmark datasets for research purposes.

2.23 Challenges and Limitations in Theory or Practice

The proposed framework faces several limitations.

- Limited historical order book datasets
- High GPU memory requirements
- Long training times
- Financial market uncertainty
- Difficulty in real-time deployment

These limitations may affect prediction performance and scalability.

2.24 Lessons from Iterative Development and Peer Feedback

The framework was improved through iterative experimentation and testing.

The improvements include:

- Hyperparameter tuning
- Architecture refinement
- Normalization improvements
- Model comparison analysis

Continuous experimentation helped improve model stability and prediction accuracy.

2.25 Responsible Use of Generative AI in Foundations Work

Generative AI tools such as ChatGPT were used responsibly for:

- Concept clarification
- Documentation support
- Theoretical content drafting
- Formatting assistance

All generated content was reviewed, modified, and validated before inclusion in the project documentation.

2.26 Summary of the Chapter

This chapter presented the foundational concepts of the proposed Mamba-LOB framework. It discussed modern algorithmic extensions, model architecture, preprocessing strategies, experimental design, evaluation metrics, implementation tools, ethical considerations, and practical limitations.

The chapter establishes the theoretical and conceptual foundation required for the implementation and experimental analysis presented in the subsequent chapters.

3 Interim Report

3.1 Chapter Overview

Briefly introduce the purpose of the interim report, scope of progress covered, and its role in shaping the remainder of the project.

3.2 Dataset Acquisition and Preprocessing

3.2.1 Dataset Description

Detail the datasets used, including source, structure, size, and features.

3.2.2 Data Quality Checks

Highlight steps taken to handle missing values, duplicates, or inconsistencies.

3.2.3 Exploratory Data Analysis (EDA)

Discuss trends, distributions, and insights from the dataset using charts and statistics.

3.2.4 Pre-processing Pipelines

Describe encoding, normalization, augmentation, or transformation techniques applied.

3.3 Feature Engineering

3.3.1 Feature Identification Techniques

Explain how features were selected or extracted from raw data.

3.3.2 Dimensionality Reduction (if applicable)

Discuss use of PCA, LDA, or other techniques to reduce feature space.

3.3.3 Feature Impact Observations

Summarize which features appear influential in preliminary testing.

3.4 Technical Implementation

3.4.1 Tools and Technology Stack

List the platforms, languages, libraries, and IDEs used so far.

3.4.2 System Setup and Environment Configuration

Explain the computing environment's OS, dependencies, GPU usage, version control.

3.4.3 Modular Code-base Description

Document implemented modules, APIs, or pipelines (with screenshots if applicable).

3.5 Baseline Model Development

3.5.1 Baseline Algorithm Description

Describe the first version of your model or algorithm.

3.5.2 Training Setup and Parameters

List hyperparameter, epochs, batch sizes, optimizers, etc., used in initial experiments.

3.5.3 Early Performance Metrics

Present results like accuracy, precision, F1-score, latency, etc., from the baseline runs.

3.5.4 Output Visualization and Interpretation

Include confusion matrices, ROC curves, or output plots to explain behavior.

3.6 Preliminary Analysis and Observations

3.6.1 Result Analysis and Discussion

Interpret model behavior and accuracy trends; identify possible sources of error.

3.6.2 Error Profiling and Bias Detection

Mention misclassifications, overfitting signs, or bias in predictions.

3.6.3 Lessons Learned

Discuss initial surprises, technical challenges, or confirmations of earlier hypotheses.

3.7 Originality and Innovation in Approach

3.7.1 Creative Framing of the Problem

Highlight unique perspectives or constraints applied in your approach.

3.7.2 Experimental Innovations

Propose alternative techniques, architectures, or data-handling strategies explored.

3.8 Risk Assessment and Project Management

3.8.1 Progress Tracker vs. Original Timeline

Show Gantt chart or milestone chart with updates.

3.8.2 Bottlenecks and Delays

Mention technical or resource-related delays and how they are being addressed.

3.8.3 Risk Mitigation Strategies

Identify potential risks in computation, ethics, or data and your contingency plans.

3.9 Experimental Reproducibility

3.9.1 Data Splits and Control Measures

Clearly state train/validation/test splits and their reproducibility strategy.

3.9.2 Random Seed Control

Mention if random seeds, initialization settings, or sampling techniques are made repeatable.

3.10 Scalability and Efficiency Considerations

3.10.1 Memory, Time, and Resource Profiling

Mention compute time, CPU/GPU usage, memory consumption.

3.10.2 Scalability Constraints

Highlight algorithmic limitations for larger datasets or real-time deployment.

3.11 Ethical Considerations and Responsible AI Use

3.11.1 Generative AI Usage Disclosure

Disclose if tools like ChatGPT, Copilot, Bard, etc., were used in writing, coding, or planning.

3.11.2 Bias and Fairness in Dataset

Comment on demographic fairness, sampling bias, or ethical concerns in your dataset or model.

3.12 Feedback Incorporation and Continuous Improvement

3.12.1 Peer and Mentor Feedback Summary

List key feedback points and how they influenced current progress.

3.12.2 Reflection and Learning Journey

Describe skills acquired, mindset changes, and personal development as a researcher.

3.13 Chapter Summary and Next Steps

Summarize interim progress, major takeaways, and refinements planned for the final phase of the project.

Optional Appendices Visuals: Confusion matrices, EDA plots, timelines

Tables: Detailed metric breakdowns, hyperparameter logs

Reproducibility: Scripts, requirements.txt, model cards

References

- [1] Antonio Briola, Silvia Bartolucci, Tomaso Aste. *Deep Limit Order Book Forecasting - A microstructural guide*.
arXiv - Quantitative Finance, 2024
- [2] Matteo Prata¹, Giuseppe Masi¹, Leonardo Berti¹, Viviana Arrigoni¹, Andrea Coletta², Irene Cannistraci¹, Svitlana Vyetrenko², Paola Velardi¹, Novella Bartolini *Lob based deep learning models for stock price trend prediction: a benchmark study*.
Artificial Intelligence Review, 2024
- [3] Adam MarszałĆek, Tadeusz BurczyÅłski, *Modeling of limit order book data with ordered fuzzy numbers*.
Applied Soft Computing, 2024
- [4] Riccardo Busetto, Simone Formentin, *Continuous-time modeling of financial returns based on Limit Order Book data*.
IFAC-PapersOnLine, 2023
- [5] Samuel Ping-Man Choi, Yin-Hei Chan, Hie-Yiin Hung, *Analyzing the Importance of Broker Identities in the Limit Order Book Through Deep Learning*
Sagepub Journals, 2021
- [6] Chiu-Hung Su, Hsu-Chao Lai, Wen-Yueh Shih, Jun-Zhe Wang, Jiun-Long Huang *Spread Movement Prediction for Pairs Trading with High-Frequency Limit Order Data*
IEEE International, 2022